System Architecture

Energy-Aware Adaptive IoT Sensing Using Edge Intelligence

1. Overview

This document specifies the system architecture for the predictive adaptive-sampling IoT node described in Deliverable 1: the physical hardware and its connections, the firmware and host-side software structure, the MQTT-based communication design, and the end-to-end data pipeline from sensor reading to the final accuracy-vs-energy trade-off curve. The architecture is designed so all three sensing policies (fixed-rate, threshold-based adaptive, and the proposed predictive-adaptive policy) run on identical hardware and share the same communication and logging path, ensuring a fair, controlled comparison.

2. Hardware Architecture

The physical system consists of a single ESP32-S3 sensing node, its attached sensor and energy-measurement circuitry, a battery power source, and a host machine running the MQTT broker and analysis software. An event inducer (a small motor for vibration sensing, or a controlled gas sample for gas sensing) is used to create repeatable, operator-timed "events" for ground-truth labeling.

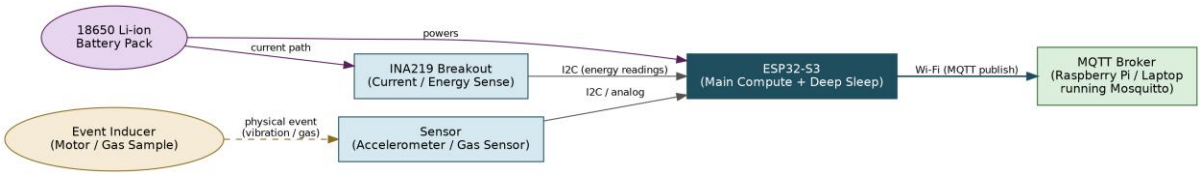


Figure 1. Hardware architecture: physical components and their connections.

Component summary

Component	Purpose
ESP32 (or ESP32-S3) development board with deep-sleep support	Main compute unit; runs all three sensing policies and adaptive logic
Accelerometer (ADXL345 or MPU6050) or gas sensor (MQ-2/MQ-135)	Primary monitored quantity — chosen for a clear, inducible stable-vs-event distinction
Current-sense resistor + INA219 breakout	Direct, ground-truth energy/current measurement during operation
Small vibration source (DC motor) or controlled gas sample	Used to induce repeatable "event" conditions on demand
18650 Li-ion battery pack and holder	Enables a realistic, hardware-based battery-life projection
Raspberry Pi or laptop running a local MQTT broker (Mosquitto)	Receives transmissions and logs data for offline analysis

3. Software Architecture

The firmware running on the ESP32 is organized into four cooperating modules: a sensor driver layer, a sensing policy engine (with three interchangeable policy implementations), a power-management module controlling deep-sleep/wake timing, and an MQTT client for publishing data. On the host side, a Python-based logging and analysis pipeline subscribes to the broker, stores readings to disk, and computes the evaluation metrics offline — including the trigger-sensitivity sweep used to tune the predictive policy.

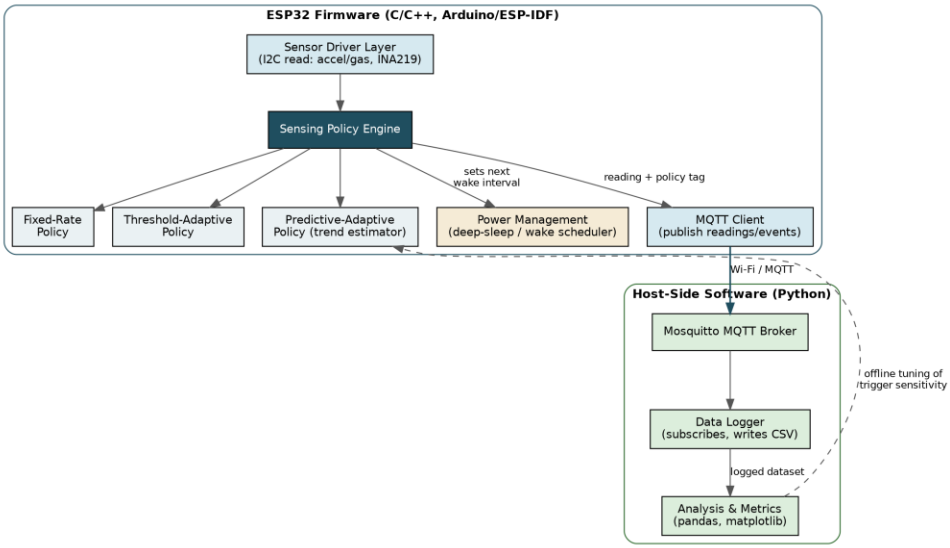


Figure 2. Software architecture: firmware modules (ESP32) and host-side analysis pipeline (Python).

Module responsibilities

- **Sensor Driver Layer:** reads the accelerometer/gas sensor and the INA219 current sensor over I2C, exposing raw readings to the policy engine.
- **Sensing Policy Engine:** selects between the fixed-rate, threshold-adaptive, and predictive-adaptive policies (only one active per experimental trial) and decides the next sampling interval.
- **Power Management:** puts the ESP32 into deep sleep between samples and wakes it according to the interval set by the active policy.
- **MQTT Client:** publishes each reading, energy value, and any ground-truth event flag to the broker over Wi-Fi.
- **Host-side Logger and Analysis:** subscribes to all topics, persists them to CSV, and computes accuracy, energy, latency, and bandwidth metrics, including the sweep used to tune predictive-trigger sensitivity.

4. Communication Architecture

Communication uses MQTT publish/subscribe over Wi-Fi, with three topics separating sensor readings, ground-truth event flags, and energy measurements. This separation lets the host-side logger reconstruct the effective sampling rate, score detection latency and accuracy against ground truth, and compute per-trial energy consumption independently for each policy under test.

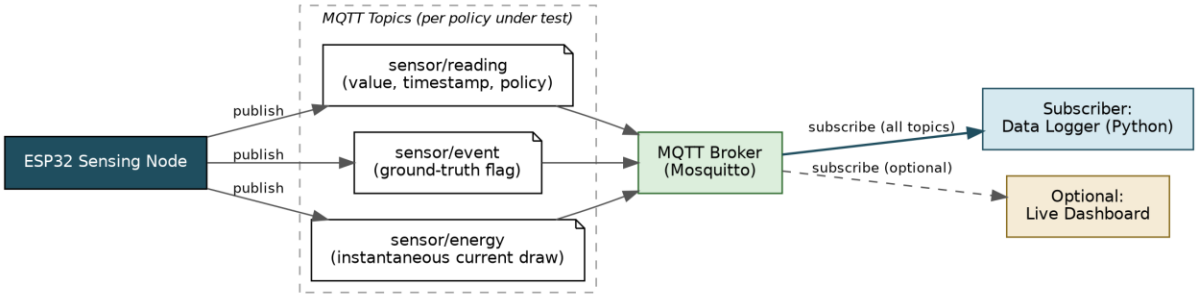


Figure 3. Communication architecture: MQTT topic design between the sensing node, broker, and subscribers.

Topic design

Topic	Payload	Publish Frequency	Purpose
sensor/reading	Sensor value, timestamp, active policy tag	Every sampling cycle (rate varies by policy)	Reconstructs the effective sampling rate per policy over time
sensor/event	Ground-truth event flag (operator-controlled induction)	On event start/end	Scores detection latency and accuracy against ground truth
sensor/energy	Instantaneous current draw from INA219	Every sampling cycle	Computes energy consumed per trial and per detected event

5. Data Pipeline

The end-to-end data pipeline runs from a single sensor reading through to the project's key experimental output: the accuracy-vs-energy trade-off curve. Stages 1–5 execute on the ESP32 in real time; stages 6–9 execute offline on the host machine after (or during) data collection.

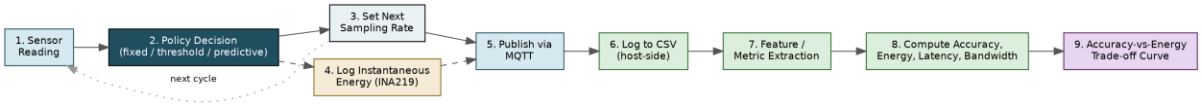


Figure 4. End-to-end data pipeline from sensor reading to trade-off curve.

Pipeline stages

Stage	Description
1. Sensor Reading	Sensor Driver Layer reads accelerometer/gas sensor over I2C/analog
2. Policy Decision	Active sensing policy (fixed-rate, threshold, or predictive) decides whether to sample now and at what rate
3. Set Next Sampling Rate	Policy updates the wake-scheduler interval for the next cycle
4. Log Instantaneous Energy	INA219 current reading captured alongside the sensor reading
5. Publish via MQTT	Reading, event flag, and energy value published to the local broker
6. Log to CSV (host-side)	Python subscriber logs all topics to a structured CSV file

Stage	Description
7. Feature / Metric Extraction	Offline processing computes per-trial features: detection timing, energy per sample, bytes transmitted
8. Compute Accuracy, Energy, Latency, Bandwidth	Metrics aggregated per policy and per predictive-trigger sensitivity setting
9. Accuracy-vs-Energy Trade-off Curve	Final sweep across trigger sensitivities plotted as the project's key result

6. Traceability to Project Objectives

This architecture directly supports the objectives set out in Deliverable 1: the shared hardware and software structure across all three policies (Section 6.6) ensures a fair comparison (Objective 2); the energy-logging path via the INA219 and the sensor/energy topic (Sections 2 and 4) supports quantifying energy consumption (Objective 4); the sensor/event topic provides the ground-truth timestamps needed to score detection latency and accuracy (Objective 3); and the offline analysis stage's sweep over predictive-trigger sensitivity (Section 5, stages 7–9) is what produces the full accuracy-vs-energy trade-off curve required by Objective 5.